DEPI | Digital Egypt Pioneers Initiative

DEPI Additional Deliverables

Project 4: SIEM Detection Engineering, Alert Triage & SOC Operations

6. Additional Deliverables – Applicability

Deliverable	Applies?	How it applies to Project 4	Timing
6.1 API Documentation	**Partly**	Uses existing Wazuh, Ollama, and VirusTotal APIs; no custom API	Weeks 3–4
6.2 Testing & Validation	**Yes**	Rule, end-to-end, and acceptance tests of the SOC workflow	Weeks 2–4
6.3 Deployment Strategy	**Yes**	Lab deployment of Wazuh; scaling covered as a design	Weeks 1 and 4

6.1 API Documentation

No custom API is built. The team uses the REST APIs of Wazuh, Ollama, and VirusTotal to check the platform, collect KPI metrics, enrich IOCs, and run the AI workflow. Wazuh paths follow the 4.x documentation.

Endpoints Used

Endpoint	Method	Purpose	Used For
Wazuh Server API (HTTPS, port 55000)			
/security/user/authenticate	**POST**	Get a JWT token for later calls	All Wazuh API calls
/manager/status	**GET**	Check that Wazuh services are running	KPI-06 availability
/agents	**GET**	List agents and their connection status	KPI-07 connectivity
/rules	**GET**	Confirm custom rules are loaded	KPI-04 coverage
Wazuh Indexer API (HTTPS, port 9200)			
/wazuh-alerts-*/_search	**POST**	Query alerts by rule ID and time	KPI-01 to 03; triage
VirusTotal API v3 (HTTPS; API key in the x-apikey header)			
/api/v3/files/{hash}	**GET**	File report by hash (MD5, SHA-1, SHA-256)	KPI-12 IOC enrichment
/api/v3/ip_addresses/{ip}	**GET**	IP address reputation report	KPI-12 IOC enrichment
/api/v3/domains/{domain}	**GET**	Domain reputation report	KPI-12 IOC enrichment
Ollama API (HTTP, port 11434)			
/api/generate	**POST**	Generate alert summaries and report drafts	AI-assisted workflow
/api/tags	**GET**	List the models installed locally	Model check

VirusTotal public API limit: 4 requests per minute and 500 per day. Look up only the IOCs from triaged alerts and save the results (risk R06).

Usage Examples

```
# 1) Get a token from the Wazuh server API
TOKEN=$(curl -sk -u <api_user>:<api_password> -X POST \
  "https://<wazuh-manager>:55000/security/user/authenticate?raw=true")
# 2) List active agents
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://<wazuh-manager>:55000/agents?status=active"
# 3) Search alerts for one rule in the Wazuh indexer
curl -sk -u <indexer_user>:<indexer_password> -H "Content-Type: application/json" \
  "https://<wazuh-indexer>:9200/wazuh-alerts-*/_search" \
  -d '{"size":10,"query":{"match":{"rule.id":"<rule_id>"}}}'
# 4) Look up an IP address in VirusTotal
curl -s -H "x-apikey: <virustotal_api_key>" \
  "https://www.virustotal.com/api/v3/ip_addresses/<ip_address>"
# 5) Ask Ollama for an alert summary (a draft only; validate it manually)
curl -s http://localhost:11434/api/generate \
  -d '{"model":"<model_name>","prompt":"Summarize this alert: <alert_text>","stream":false}'
```

Replace the values in angle brackets. The -k option skips certificate checks and is acceptable only inside the isolated lab.

API Security Notes

- Keep API usernames, passwords, tokens, and the VirusTotal API key out of GitHub; store them in local environment variables or a private file.
- Use lab accounts, with read-only access where possible, for the metric and reporting calls.
- Send only indicators (hashes, IPs, domains) to VirusTotal. Files and URLs submitted to it become visible to other users, so never upload sensitive files.
- Keep the Wazuh and indexer APIs on the lab network only. Ollama listens on the local machine by default and has no login, so do not expose it.

6.2 Testing & Validation

Pass criteria use the KPI targets. Results are saved in the tuning notes and test results deliverable on GitHub.

Test Approach

Level	What Is Tested	Method	Pass Criteria
Unit (rule level)	Each rule and decoder on its own	Sample logs run through wazuh-logtest	Expected rule ID and level; no match on normal events
Integration	Endpoint to alert, dashboard, enrichment, and AI summary	Controlled test events in the lab	Alert appears; KPI targets met (e.g., MTTD ≤ 5 min)
User acceptance (UAT)	The SOC workflow as used by analysts	Team acts as analysts; instructor review	Reports and playbook completed without help; instructor accepts

Test Cases

ID	Test	Expected Result
Unit Tests (Week 2)		
UT-01	Brute force rule with repeated failed logins in sample logs	Rule fires after the threshold; one failure does not
UT-02	Suspicious login rule with unusual login samples	Rule fires with the expected level
UT-03	Lateral movement rule with sample logs	Rule fires on movement between systems
UT-04	Persistence rule with sample logs (new service, scheduled task)	Rule fires when the telemetry supports it
UT-05	Data exfiltration rule with sample outbound transfer logs	Rule fires on the suspicious transfer pattern
UT-06	Decoders and field mapping for each onboarded source	Normalized fields are extracted correctly
Integration Tests (Weeks 2–3)		
IT-01	Agents connect to the manager	All onboarded agents show Active
IT-02	End-to-end detection of a controlled endpoint event	Alert appears in the dashboard within the MTTD target
IT-03	Dashboard with real alerts	All 6 required metrics show data
IT-04	VirusTotal lookup for an IOC from a test alert	Enrichment result is returned and recorded
IT-05	Ollama summary of a test alert	Summary is generated and checked manually against the evidence
User Acceptance Tests (Weeks 3–4)		
UAT-01	Triage of 3 sample alerts using the triage workflow	3 complete triage reports with conclusions
UAT-02	Team member follows the SOAR playbook (phishing or malware)	All steps can be completed without extra help
UAT-03	Demo and deliverables review with the instructor	Instructor accepts the deliverables

6.3 Deployment Strategy

The project runs as an isolated lab, not a production system. Whether the VMs run locally or in the cloud is decided in Week 1 (risk R02).

Hosting Environment

Component	Role	Hosting Notes
Wazuh server	Manager, indexer, and dashboard (all-in-one)	One Linux VM sized to the official Wazuh requirements
Monitored endpoints	Log sources with Wazuh agents installed	Lab VMs chosen in the data source inventory
Ollama	Local AI for alert summaries and report drafts	Lab machine with enough memory for the chosen model
VirusTotal API	Threat intelligence lookups for IOCs	External cloud service; needs an API key and internet access
GitHub repository	Rules, configuration, and documents	DEPIIRTEAM/DEPI_PROJECT4
Lab network	Isolated network for all test events	Attack simulations stay inside this network

Deployment Process

Step	Action	How	Check
1	**Prepare**	Edit the rule or configuration and save it in GitHub	XML is valid
2	**Test**	Run sample logs through wazuh-logtest	Expected rule ID fires
3	**Deploy**	Copy rules to /var/ossec/etc/rules/local_rules.xml and restart the Wazuh manager	Manager starts without errors
4	**Verify**	Run the integration test with a controlled event	Alert appears in the dashboard
5	**Record**	Commit and tag the version in GitHub with the test results	Version is tagged
6	**Roll back (if needed)**	Restore the previous tagged version or the VM snapshot	Manager runs with the previous rules

Take a VM snapshot before every major change. Optionally, a GitHub Actions workflow can check the rule files for XML errors automatically.

Scaling Considerations

Area	Lab (This Project)	Medium-Sized Organization (Design)
Wazuh manager	Single node	Manager cluster (master + workers) with a load balancer
Indexer and storage	Single node; retention from the retention plan	Multi-node cluster; storage from log volume × retention
Agents	A few lab endpoints	Agent groups with central configuration
AI (Ollama)	Small local model	Dedicated host sized for the chosen model
Backup	VM snapshots and GitHub	Scheduled backups of configuration and indices